



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

ERASMUS MUNDUS JOINT MASTER IN
INTELLIGENT FIELD ROBOTIC SYSTEMS

LiDAR-Based Mapping Module For Autonomous Navigation

Author:

Zewdie-Habtie Sisay

MSc Intelligent Field Robotic Systems (IFRoS)

Internal supervisor:

Zoltán Istenes

Professor

External supervisor:

Mohammad Albaja

Associate Professor, SMART

Budapest, 2025

Acknowledgements

Contents

Acknowledgements	1
1 Introduction	3
1.1 Background	3
1.1.1 Online SLAM	4
1.1.2 Full SLAM	5
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Thesis structure	6
2 Literature Review	7
2.1 Related works	8
3 Methodology	10
3.1 Introduction to joint probability	10
3.1.1 State estimation??	10
3.2 System Overview	10
3.2.1 FAST-LIO2	11
3.2.2 Loop detection	11
3.2.3 Loop closure	11
3.2.4 Optimization	11
4 Experiments and Results	12
4.1 Dataset	12
4.1.1 Analysis of Accuracy	13
4.2 Dynamic Objects removal	14
5 Conclusion	15

CONTENTS

Bibliography	16
List of Figures	17

Chapter 1

Introduction

1.1 Background

In recent years, robotics has evolved with the goal to develop intelligent robots that can assist or replace human beings in performing tasks. In order for these robots to operate effectively and efficiently in diverse environment, a set of modules that define the robot autonomy such as path planning, localization, mapping, perception, and manipulation are required. These modules are interconnected to each other in a sense that one can be an input to the other. In similar manner to how we humans understand and interact with our environment, robots also need to have these modules tightly integrated.

Map is an important prerequisite that helps a robot determine the pose of objects within its environment, enabling it to operate and perform tasks such as navigation, obstacle avoidance, and path planning. Both mapping and localization modules are usually combined and referred to as Simultaneous Localization and Mapping (SLAM) module where the robot builds the map while continuously localizing itself inside it. The planning module allows the robot to plan a path from its current pose (position, and orientation) to a goal pose. An algorithm that assists the robot to follow the planned path, a controller, is also considerably essential entity. Using perception module, robot senses and interpret the environment. In same way we humans use our sensing organs to sense our surrounding, robot uses its onboard sensors such as camera, Light Detection and Ranging (LiDAR), Radio Detection and Ranging (RADAR), Inertial Measurement Unit (IMU), Global Positioning System (GPS) and other sensors. The manipulation module enables the robot to interact with objects in its environment by performing tasks such as

grasping, picking, placing, and etc.

Map being an important prerequisite and SLAM being the state-of-the-art solution to generating it, there are several techniques to solving SLAM in the literature using different or similar methods. The two main techniques are online SLAM and full SLAM. These techniques rely on different sensor modalities with LiDAR SLAM and Visual SLAM being two the most widely used approaches.

LiDAR SLAM uses laser sensors to measure distances and create precise 3D maps, making it highly effective in dark or low-feature environments. However, its high cost and power consumption make it less suitable for low-cost robotics. On the other hand, Visual SLAM (V-SLAM) depends on cameras (monocular, stereo, or RGB-D) to detect visual features and track motion, providing a more affordable and lightweight solution. While V-SLAM captures detailed textures and colors, it struggles in low-light conditions and areas with few visual features. Despite these differences, both types are essential in robotics and autonomous systems. To improve accuracy, many modern applications combine both techniques using sensor fusion, taking advantage of their strengths to enhance localization and mapping performance. Based on the solution methodology used, SLAM can be further categorized into online SLAM and Full SLAM.

1.1.1 Online SLAM

Online SLAM methods estimate the current state x_t and the whole map m based on all past observations $z_{1:t}$ and controls $u_{1:t}$. These methods use Kalman Filter(KF), Extended Kalman Filter (EKF), Particle Filter (PF), and Unscented Kalman Filter (UKF)[1]. EKF-SLAM is one of the most popular online SLAM algorithms that uses Bayes filter to estimate the robot poses and landmark locations, assuming Gaussian distributions and local linearity. Bayes filter is a recursive filter for state estimation that includes two steps, prediction and update as shown in Eq (1.1), where $\bar{belief}(x_t)$ represents the prior belief about the robot state at time t before incorporating the latest observation (prediction step) and $belief(x_t)$ is a posterior belief that defines the robots refined state after incorporating latest observation (update step).

$$\begin{aligned}\bar{belief}(x_t) &= \int p(x_t|x_{t-1}, u_t) belief(x_{t-1}) dx_{t-1}, \\ belief(x_t) &= p(z_t|x_t)\bar{belief}(x_t).\end{aligned}\tag{1.1}$$

Map generated with online slam is not accurate due to the linearization errors accumulated over time. Hence full SLAM solution mitigates this drawback of online SLAM.

1.1.2 Full SLAM

Full SLAM aims to solve the problem of finding the most probable sequence of robot poses $x_{1:t}$ and the map m , given all sensor measurements $z_{1:t}$ and control inputs $u_{1:t}$ [2].

$$p(x_{1:t}, m | z_{1:t}, u_{1:t}) \propto p(x_{0:t})p(m) \prod_{t=1}^T p(z_t | x_t, m)p(x_t | x_{t-1}, u_t) \quad (1.2)$$

where $p(x_{1:t})$ represents the prior on the robot motion, $p(m)$ is prior on the map, $p(z_t | x_t, m)$ is the likelihood of observations given the map and pose, and $p(x_t | x_{t-1}, u_t)$ describes the motion model[3]. Full SLAM method is mostly used to build consistent map using point clouds and solve structure from motion problems, specially to handle loop closure events.

Bayesian network

Factor Graph

One approach to solving the Full SLAM problem is using Bayesian network where the problem is formulated as joint distribution of all the variables ($x_{1:t}$, $z_{1:t}$, $u_{1:t}$, and m). However, the Bayesian network uses only arrows to define dependency between variables which does not encode much of information.

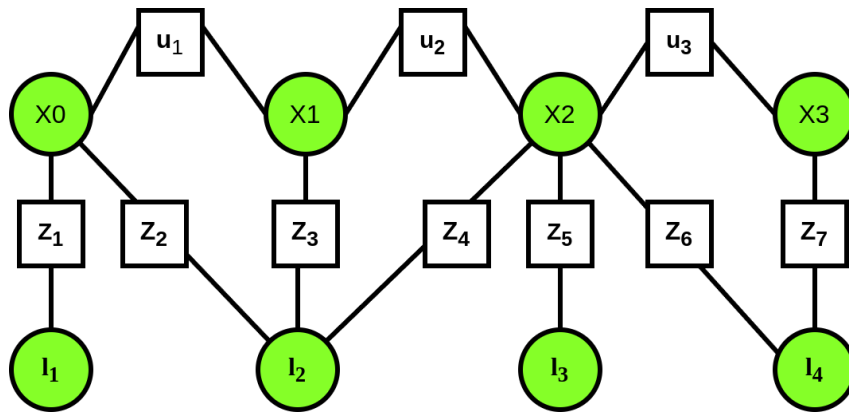


Figure 1.1: Full SLAM model with factor graph

Another common approach to solving Full SLAM is using Factor Graphs, which represent the probability distribution as a bipartite graph with variable nodes (green circles in Fig.

1.1) and factor nodes (white squares in Fig. 1.1) where edges can only connect variable nodes via factor nodes. We denote variable nodes as $\mathbf{x}$ and factor nodes as ϕ . The factors define the probabilistic constraint motion model and the likelihoods measurements. The joint probability distribution of the problem is then computed as the product of all factors in the graph (Eq. 1.3). This graph structure allows efficient optimization via methods such as Gaussian Belief Propagation or Nonlinear Least Squares Optimization.

$$P(\mathbf{x}, \mathbf{z}) \propto \prod_{k=1}^K \phi_k \quad \text{where } \phi_k \geq 0 \quad \forall K \quad (1.3)$$

1.2 Problem Statement

1.3 Objectives

1.4 Thesis structure

Chapter 2

Literature Review

The goal of this chapter is to present the research context in which our project was conducted, by looking at common methods for each of the main components and highlighting those that inspired the current approach.

In mobile robotics, map is an important prerequisite that enables robots to operate and perform different tasks such as navigation, obstacle avoidance, path planning, etc. SLAM is the most popular method in the literature to generate maps. Among the different methods of SLAM, vision-based and LiDAR technologies are the most popular in the literature. Although each SLAM method has its advantages and drawbacks, they are essential for different tasks. LiDAR technologies will likely remain essential for years to come due to their numerous advantages: robust to various lighting conditions, accurate 3D measurements with higher Field of View (FoV), among others, despite their high cost and bulkiness to install in small-scale robots like drones. Solid-state LiDARs are cost-effective, lightweight, and precise, making them ideal for UAVs in mapping and complex environments. Although these solid-state LiDARs have good potential, they have presented new challenges to SLAM solutions: 1) When the robot is operating in a cluttered environment, there not be strong features with geometric shapes like edges and planes. This causes the LiDAR SLAM to degenerate easily. 2) Fusing many feature points with IMU is challenging. 3) LiDAR points are sequentially sampled resulting in each point having a different timestamp which causes motion distortion that severely affects the point registration [4].

Several works exist in the literature on LiDAR SLAM. Some of these works use filter-based (Kalman filter) approach while others use optimization-based(factor graphs)

for the state estimation. Although filter-based approaches are simple in terms of implementation and computational requirements, they accumulate errors over time causing a larger drift. Optimization-based approaches on the other hand formulate the SLAM problem as non-linear problem and optimize the whole robot trajectory and the map minimizing the drift, more specifically during loop closure. Here we review the most related and relevant works.

2.1 Related works

[5] proposed a real-time method for odometry and mapping. They achieved low-drift results without the need to use IMU and high-accuracy range measurements. The costly SLAM problem is divided into two algorithms where one algorithm computes the odometry at a higher frequency and the other algorithm runs at a much lower frequency to register point cloud.

A method by [4] presented a novel approach to tightly-coupled LiDAR-inertial odometry by proposing a new formula for computing the Kalman gain, which demonstrates equivalence to the conventional formula but reduces computation complexity based on state dimensions rather than measurement dimensions. An Iterative Kalman Filter(IKF) is used to minimize the point-to-plane distance during the registration of scan to map. The authors implemented their method into a robust LiDAR-inertial odometry package called FAST-LIO that operates effectively on a small-scale quadrotor. Experiments conducted in varied environments validate the system’s resilience against fast motion and vibration noise.

[6] presented Scan Context, a novel spatial descriptor designed for outdoor place recognition using 3D LiDAR scans. Unlike traditional histogram-based methods, Scan Context encodes the entire point cloud into a matrix format, preserving the absolute geometrical structure of the environment. This approach utilizes a cosine distance measure for similarity scoring and implements a two-phase search algorithm to enhance loop detection efficiency, particularly in urban settings. Experimental results demonstrate that Scan Context significantly outperforms existing global descriptors in various datasets, providing robust performance against noise and viewpoint changes.

[7] present a novel approach for large-scale LiDAR mapping by introducing a hierarchical LiDAR bundle adjustment (HLBA) framework. The method aims to improve the consistency and accuracy of LiDAR-based maps by formulating a global optimization problem that jointly refines multiple scans. Unlike conventional SLAM techniques that rely on pairwise constraints, HLBA leverages a hierarchical structure to balance local precision and global consistency, mitigating drift accumulation over extended trajectories. The authors demonstrate the effectiveness of their approach through extensive experiments on real-world datasets, showcasing its advantages in handling large-scale environments with minimal loop closures. Their results indicate that HLBA outperforms existing methods in terms of mapping accuracy and robustness, particularly in scenarios with sparse revisits.

A key contribution of this work is the introduction of an efficient optimization strategy that scales well with the number of LiDAR scans. By organizing the LiDAR data into hierarchical layers, the proposed framework enables efficient refinement without requiring excessive computational resources. The authors also incorporate a robust outlier rejection mechanism to enhance data association, ensuring reliable pose estimation in challenging conditions. Furthermore, the method integrates seamlessly with existing LiDAR-based odometry pipelines, making it adaptable to various robotic platforms. Overall, this study provides a significant advancement in large-scale LiDAR mapping, offering a practical solution for long-term autonomous navigation and high-fidelity 3D reconstruction.

[8] significantly improved upon the original work [4] by introducing two key advancements: direct point registration and the incremental k-D tree (*ikd-Tree*). Unlike [4], which relies on hand-crafted feature extraction, the new approach registers raw LiDAR points directly to the map, increasing accuracy and adaptability to different LiDAR sensors. The newly introduced *ikdTree* enables efficient incremental updates, dynamic re-balancing, and on-tree downsampling, reducing computational load while maintaining real-time performance. These improvements allow a new approach to achieve higher accuracy at a lower computational cost, making it more robust for diverse environments, including those with small FoV LiDARs and aggressive motion.

As showing Figure ?? and Figure ??, *ikd-Tree* data structure achieves best overall performance both for point insertion time and point search time.

Chapter 3

Methodology

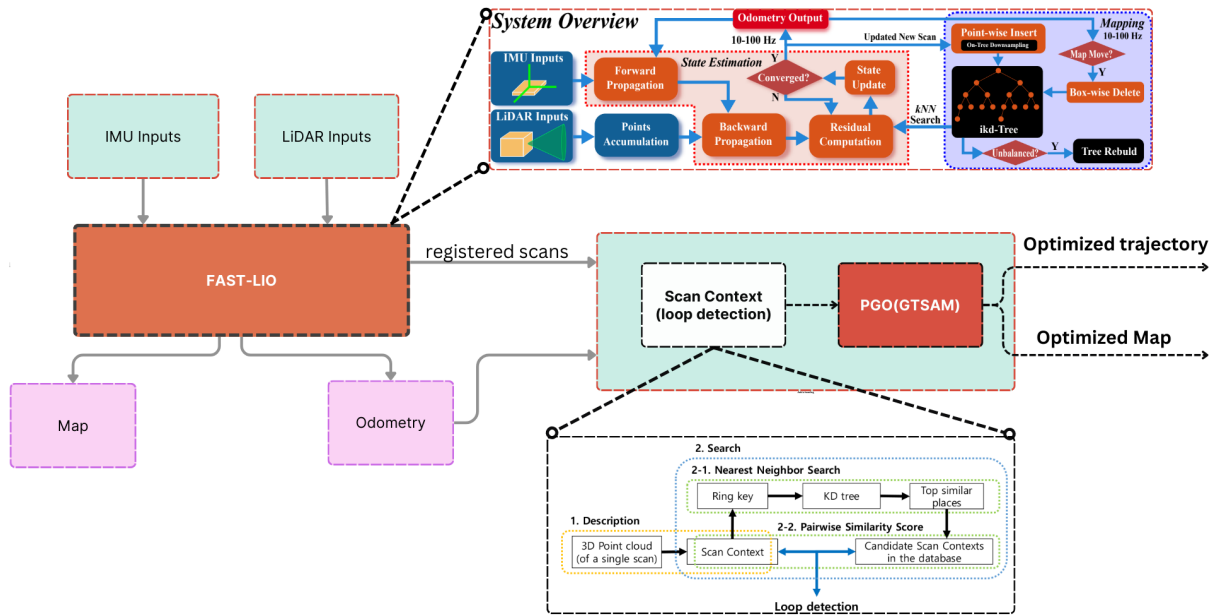


Figure 3.1: System overview of FAST-LIO with loop closure detection and optimization.

3.1 Introduction to joint probability

3.1.1 State estimation??

3.2 System Overview

The diagram illustrates a LiDAR-based FAST-LIO mapping system with loop closure detection and pose graph optimization. The key components include:

- **FAST-LIO**: This module integrates IMU and LiDAR data to estimate the system's state in real time. It employs forward propagation for state prediction and backward propagation for refinement. Point cloud data is accumulated and processed using kNN search and residual computation to update the system state. The odometry output is generated at 10-100 Hz, while mapping is performed using an ikd-Tree for efficient point-wise insertion and deletion.
- **Scan Context**: This block is responsible for detecting loop closures by comparing newly acquired scans with previously stored scan contexts. It involves two main steps: (1) Nearest Neighbor Search, where a KD-tree and ring key are used to retrieve the most similar scan locations, and (2) Pairwise Similarity Score computation, which evaluates the resemblance between scan contexts to identify loop closures.
- **PGO (GTSAM)**: Optimizes the detected loop closures to refine the trajectory and map.
- The final output is an optimized trajectory and an improved map representation.

3.2.1 FAST-LIO2

3.2.2 Loop detection

Scan context establishes compact representation by capturing highest points from the structure in the absence severe disturbances in the roll-pitch directions.

3.2.3 Loop closure

3.2.4 Optimization

Chapter 4

Experiments and Results

4.1 Dataset

put description of the dataset and the sensors used in each and parameters things like clearly.

To evaluate the performance of our mapping module, we performed experiments with various open source dataset and our own dataset collected around Saxion building. The experiments performed with the open source dataset were performed to evaluate the robustness of our mapping module to different variety of sensors types, seasonal and environmental changes. opensource dataset contains LiDAR data, IMU data, gps data, ZED camera data. the LiDAR type is Ouster with 128 scan lines and 360deg FoV. The rate of the IMU, and LiDAR

Table 4.1: Dataset used for evaluating performance of our mapping module

Dataset	LiDAR	IMU	GPS	Ground Truth	Size	Duration(s)
Saxion-sidewalk	OS-128	xsens	CL	✗	15GB	???
MulRan-KAIST02	OS-64	xsens	CL	✓	23GB	???
HeLiPR DCC06	OS-128	RTK	RTK	✓	47GB	1074
KITTI 183	V-64??	???	CL	✓	18GB	???

In order for the SLAM to work correctly for each dataset, an extrinsic calibration between the IMU and LiDAR sensors is required. Table 4.1 the calibration matrix for each dataset used in the experiments.

4.1.1 Analysis of Accuracy

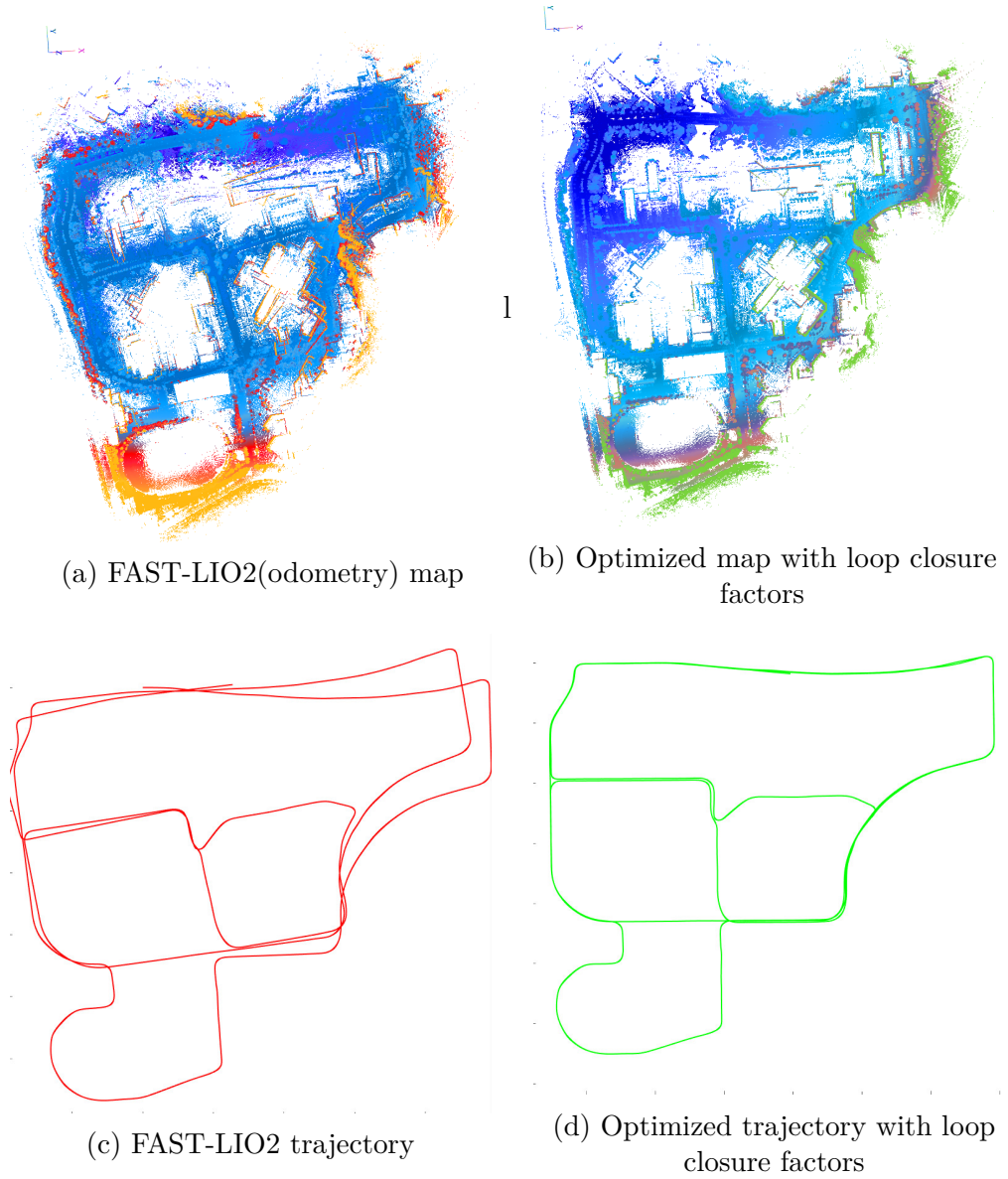


Figure 4.1: Map and path comparison of odometry vs factor graph for MulRan KAIST02 sequence

Figure 4.1 shows the result of the map and trajectory before and after applying optimization to the output of the LiDAR odometry based poses. It can be clearly seen in 4.2b that the map generated by odometry is not accurate and it has duplication in most part of the map specially on the top right corner side.

As initial step, we tested on the Mulran KAIST02 sequence using the odometry from FAST-LIO2 and generated map both from the odometry and pose-graph optimization.

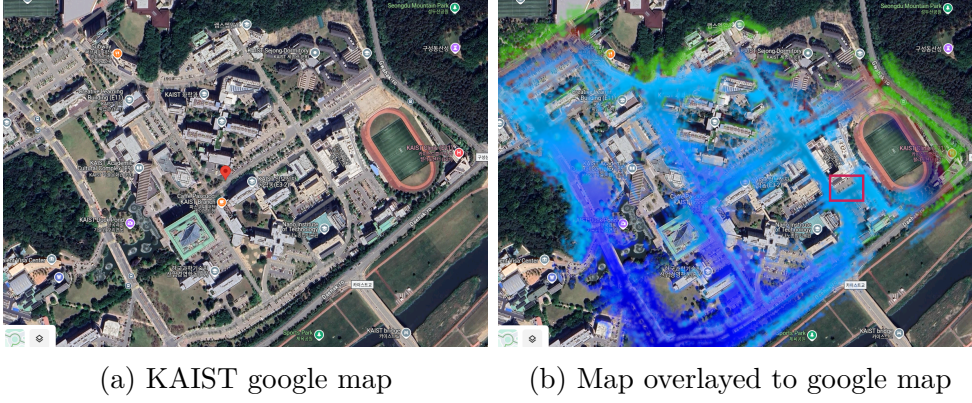


Figure 4.2: Odometry showing direction of driving(not fully optimized because both optimization and publishing the odometry run in separate threads so the published odometry is not final optimized result)

The map shown in 4.1b is generated after the FAST-LIO2 poses are optimized by ISAM2 after adding loop closure constraints to the factor graph. Since this map is generated from the optimized poses, the duplications are now removed and is more consistent than the previous map. The drift accumulated by FAST-LIO2 odometry can be seen in figure 4.1c and figure 4.1d shows the trajectory after the drift is corrected with ISAM2.

To further analyze the accuracy of the optimized map, we overlayed the generated map onto the google map, and it can be seen that some small inconsistencies are visible because unlike LiDAR bundle adjustment methods [bibid], pose graph optimization does not optimize point cloud consistency directly[9]. Hence the divergence is not completely eliminated in the map.

4.2 Dynamic Objects removal

TODO!!!!

Chapter 5

Conclusion

Bibliography

- [1] Guoquan P Huang, Anastasios I Mourikis, and Stergios I Roumeliotis. “Analysis and improvement of the consistency of extended Kalman filter based SLAM”. In: *2008 IEEE International Conference on Robotics and Automation*. IEEE. 2008, pp. 473–479.
- [2] Sebastian Thrun. “Probabilistic robotics”. In: *Communications of the ACM* 45.3 (2002), pp. 52–57.
- [3] Pere Ridao and Roger Pi. *prpy: Python Library*. Oct. 2023.
- [4] Wen Xu et al. “FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-coupled Iterated Kalman Filter”. In: *IEEE Robotics and Automation Letters* 6 (2021), pp. 7270–7276.
- [5] Ji Zhang, Sanjiv Singh, et al. “LOAM: Lidar odometry and mapping in real-time.” In: *Robotics: Science and systems*. Vol. 2. Berkeley, CA. 2014, pp. 1–9.
- [6] Giseop Kim and Ayoung Kim. “Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2018, pp. 4802–4809.
- [7] Xiyuan Liu et al. “Large-Scale LiDAR Consistent Mapping Using Hierarchical LiDAR Bundle Adjustment”. In: *IEEE Robotics and Automation Letters* 8.3 (2023), pp. 1523–1530. DOI: 10.1109/LRA.2023.3238902.
- [8] Wei Xu et al. “Fast-lid2: Fast direct lidar-inertial odometry”. In: *IEEE Transactions on Robotics* 38.4 (2022), pp. 2053–2073.
- [9] Yue Pan et al. “MULLS: Versatile LiDAR SLAM via Multi-metric Linear Least Square”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2021.

List of Figures

1.1	Full SLAM model with factor graph	5
3.1	System overview of FAST-LIO with loop closure detection and optimization.	10
4.1	Map and path comparison of odometry vs factor graph for MulRan KAIST02 sequence	13
4.2	Odometry showing direction of driving(not fully optimized because both optimization and publishing the odometry run in separate threads so the published odometry is not final optimized result)	14

